

Heuristic Search

Ramon Fraga Pereira

Sapienza Università di Roma, Italia

MSc in Engineering in Computer Science

May, 2022



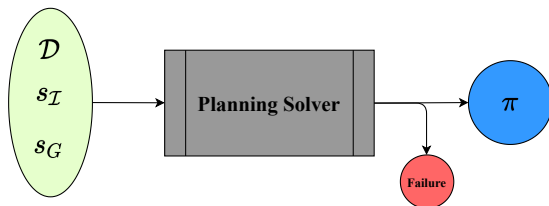
SAPIENZA
UNIVERSITÀ DI ROMA

Learn About:

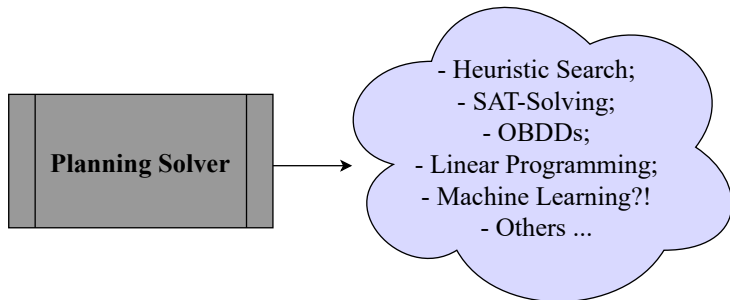
- Deterministic Heuristic Search: A^* ;
- Non-Deterministic Heuristic Search: AO^* ;

AI Planning

- In **AI Planning**, a *plan* π is synthesized from a *domain model* $\mathcal{D}$, an *initial state* s_I and a *goal (condition) state* s_G ;
- A *domain model* $\mathcal{D}$ defines the **properties** and **actions** of a particular environment.



- How to solve **AI Planning**?!



- In this lecture, we are going to discuss **Heuristic Search for Deterministic and Non-Deterministic Environments**.

AI Planning Applications¹

- Warehouse Logistics;
- Robotics;
- Agriculture;
- Underwater Operations;
- Space Operations;

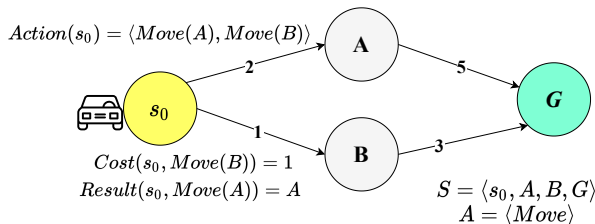


¹ <http://aiplan4eu-project.eu/>

Informed and Uniformed Search

State-Space Search Problems

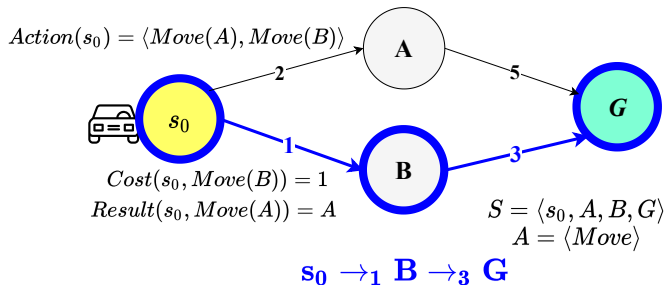
- *State-space search problems*² are among the “simplest” and most important classes of AI problems.
 - S is a set of **possible states**;
 - A is a set of **possible actions**;
 - $Action(s)$ defines that **applicable actions** in s ;
 - $Result(s, a)$ returns the **resulting state** when performing a in s ;
 - $Cost(s, a)$ is the **cost** of performing a in s .



² *State-space search problems* are **P-SPACE Complete**.

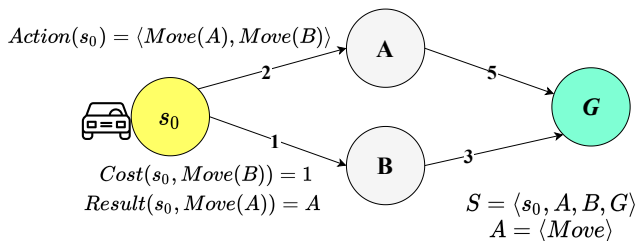
State-Space Search Problems

- **Solution:** A sequence of actions (a plan) that achieves a goal state G from an *initial state* s_0 ;
- **Performance measure:** minimize the total cost of *plan*.



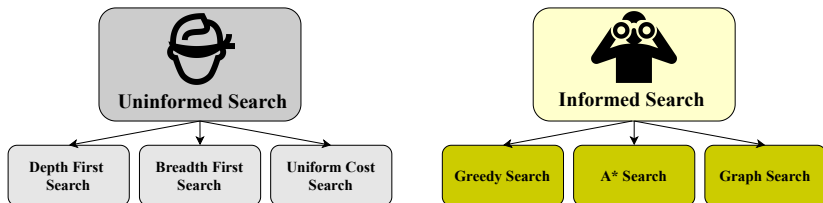
State-Space Search Notation

- States = Nodes (Vertices);
 - s_0 , A, B, G;
- Actions = Edges (Links);
 - $s_0 \rightarrow_2 A$;
 - $s_0 \rightarrow_1 B$;
 - $A \rightarrow_5 G$;
 - $B \rightarrow_3 A$;



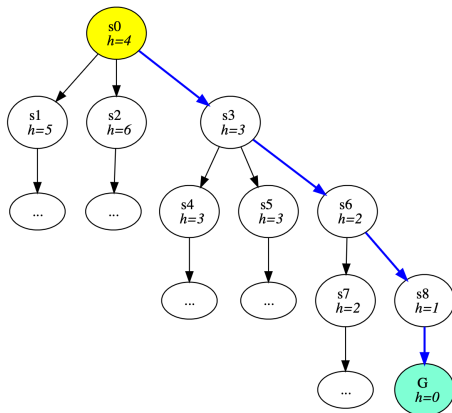
Search Algorithms: Uninformed vs. Informed Search

- **Uninformed Search** (*Blind Search*) is a searching algorithm that uses no information about from the current state s to the goal G .
- **Informed Search** (*Heuristic Search*) is a searching algorithm that uses information about the estimated distance from the current state s to the goal G .



(Informed) Heuristic Search

- *Heuristic Search* is a type of algorithm in which the goal state is used to **guide the search** by means of **evaluation functions** $f(s)$;
- **Heuristic functions** $h(s)$ estimate the cost to achieve the goals state G from a state s ;



Heuristic Functions

- An **admissible heuristic** never overestimates the cost of reaching the goal state;
 - $h(s)$ is **admissible** if, $\forall s \in S$:
 - $h(s) \leq h^*(s)$, where $h^*(s)$ is the **optimal** cost to achieve the goal.
- A **non-admissible heuristic** overestimates the cost to achieve the goal state;
- For example:
 - $h^*(Rome, Naples) = 200km$, the **optimal** cost from *Rome* to *Naples*.
 - $h_1(Rome, Naples) = 150km$, h_1 is **admissible**;
 - $h_1(Rome, Naples) = 199km$, h_1 is **admissible**;
 - $h_2(Rome, Naples) = 201km$, h_2 is **non-admissible**;
 - $h_2(Rome, Naples) = 300km$, h_2 is **non-admissible**;

- How to design informative (near optimal, admissible) heuristics?
 - **Domain-Dependent Heuristics** are usually “easier” to design;
 - **Domain-Independent Heuristics** are “more difficult” to design;

Heuristic Search: Direction

- How to apply *Heuristic Search* to solve a problem?
It depends on the **search direction**.
 - **Progressive Search:** *Forward* from initial state s_0 to a goal state G ;
 - **Regressive Search:** *Backward* from goal state G to initial state s_0 ;
 - **Bidirectional Search:** Searching *forward* and *backward* to “meet” in the middle;

In this lecture, we study **Progressive (Forward) Heuristic Search**.

Deterministic Heuristic Search (A^*)

A* Search

- A* is one of the most famous search algorithms in the literature³;
- A* performs **heuristic search by evaluating states** using f :
 - $f(s) = h(s) + g(s)$
 - $h(s)$ **estimated cost** to G from s ;
 - $g(s)$ **cost** from the initial state s_0 to s .
- $f(s) =$ **estimated cost of the cheapest plan through s .**
- **Input:** a state-space search problem;
- **Output:** a solution (a plan), or failure;

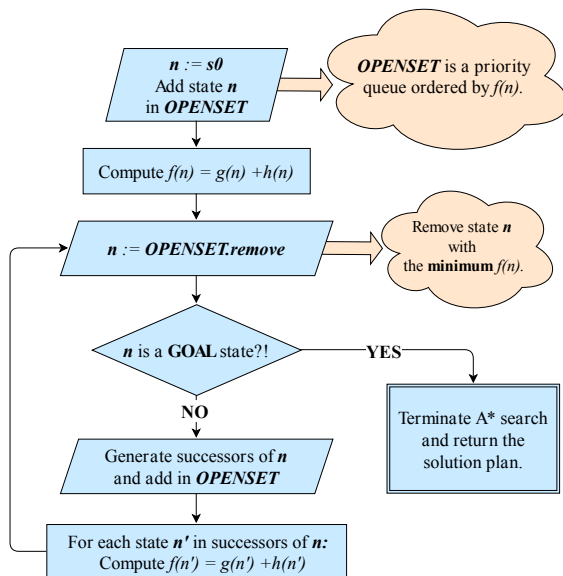
³ Hart, P. E.; Nilsson, N. J.; Raphael, B. A Formal Basis for the Heuristic Determination of Minimum Cost Paths. *IEEE Transactions on Systems Science and Cybernetics*. 1968.

Assumptions of A* Search

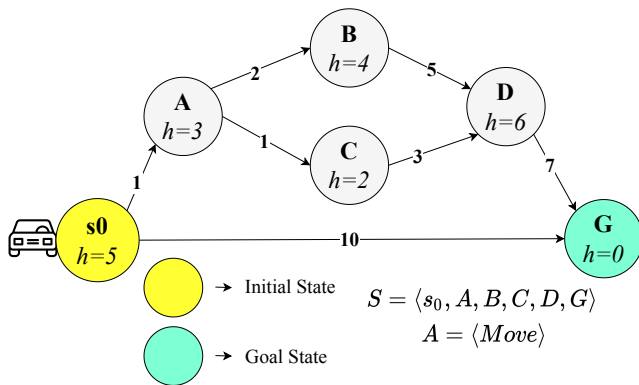
- **Single-agent**⁴;
- The states are **fully-observable** of the environment are known;
- **Finite number** of states S and actions A ;
- Actions have **deterministic effects**;

⁴ A* can be adapted to multiple agents.

A* Search Algorithm

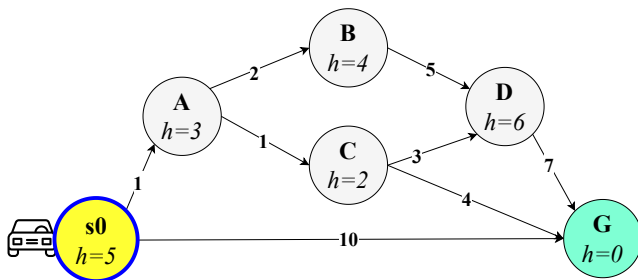


A* Search Example (1 of 5)



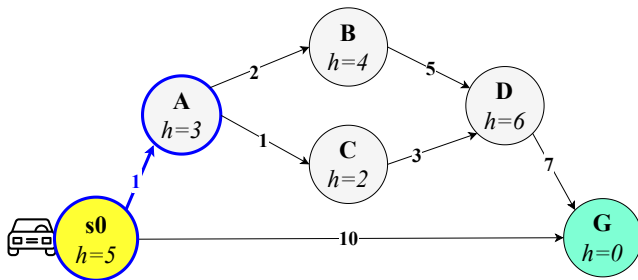
A* Search Example (2 of 5)

- $OPENSET = \langle s_0 \rangle$
 - $f(s_0) = g(s_0) + h(s_0) = 0 + 5 = 5$
 - **Remove** from $OPENSET$ the state with **minimum** $f(s)$: s_0
 - **Add successors** of s_0 in $OPENSET$:
 - $OPENSET = \langle \underline{A}, \underline{G} \rangle$



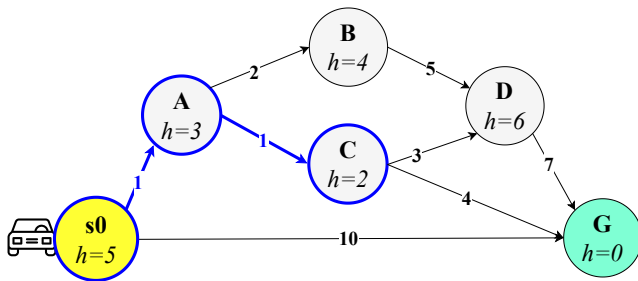
A* Search Example (3 of 5)

- $OPENSET = \langle A, G \rangle$
 - $f(A) = g(A) + h(A) = 1 + 3 = 4$
 - $f(G) = g(G) + h(G) = 10 + 0 = 10$
 - **Remove** from $OPENSET$ the state with **minimum** $f(s)$: **A**
 - **Add successors** of **A** in $OPENSET$:
 - $OPENSET = \langle \underline{C}, \underline{B}, G \rangle$



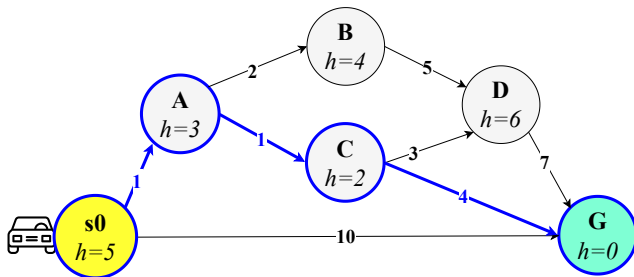
A* Search Example (4 of 5)

- $OPENSET = \langle C, B, G \rangle$
 - $f(C) = g(C) + h(C) = 2 + 2 = 4$
 - $f(B) = g(B) + h(B) = 3 + 4 = 7$
 - $f(G) = g(G) + h(G) = 10 + 0 = 10$
 - **Remove** from $OPENSET$ the state with **minimum** $f(s)$: **C**
 - **Add successors** of **C** in $OPENSET$:
 - $OPENSET = \langle \underline{G}, B, \underline{D} \rangle$



A* Search Example (5 of 5)

- $OPENSET = \langle G, B, D \rangle$
 - $f(G) = g(G) + h(G) = 6 + 0 = \mathbf{6}$
 - $f(B) = g(B) + h(B) = 3 + 4 = \mathbf{7}$
 - $f(D) = g(D) + h(D) = 5 + 6 = \mathbf{11}$
 - **Remove** from $OPENSET$ the state with **minimum** $f(s)$: G
 - G is the goal state, then return **solution**:
 - $s_0 \rightarrow_1 A \rightarrow_1 C \rightarrow_4 G$



Theoretical Properties of A* Search

Theorem (Termination and Completeness)

A search algorithm is complete and terminate if:*

- 1 *The branching factor is finite; and*
- 2 *The action costs are non-negative.*

Theorem (Optimal Efficiency)

A search algorithm is optimal if:*

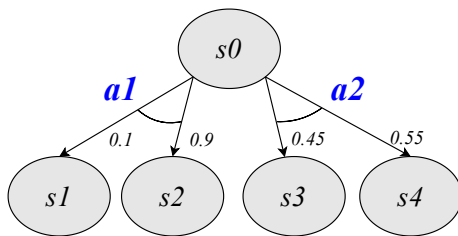
- 1 **Admissibility:** *$h(s)$ must be an admissible heuristic; and*
 - 2 **Consistency:** *h must be consistent (monotone).*
- **Space Complexity:** $O(b^d)$, where b is the **branching factor**, and d is the **depth**.

- We studied *Heuristic Search* for **deterministic** environments;
 - A* is very effective if we have a “**good**” heuristic function h .
 - Are there other Heuristic Search algorithms?
Yes, Weighted A*, IDA*, Greedy Best-First Search, others.
- Can we do *Heuristic Search* in **non-deterministic/stochastic** environments?!
 - **AND/OR Heuristic Search (AO*)**.

Non-Deterministic Heuristic Search (AO*)

AND/OR Heuristic Search

- AND/OR Heuristic Search⁵ can solve **non-deterministic** state-space search problems;
 - Each possible outcome occurs with some **probability** $P(s'|s, a)$;
- AND/OR Heuristic Search⁶ is usually used to solve:
 - *Stochastic Shortest-Path problems*, MDP problems with terminal states;
 - FOND (Fully-Observable Non-Deterministic) planning problems;

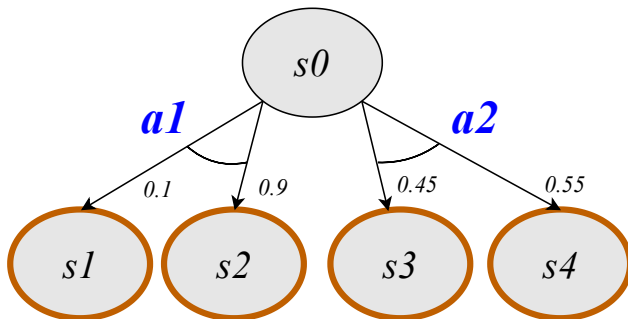


⁵ Martelli, A. and Montanari, U. 1978. *Optimizing Decision Trees Through Heuristically Guided Search*. *Communications of the ACM* 21(12):1025-1039.

⁶ Nilsson, N.J. 1980. *Principles of Artificial Intelligence*.

AND/OR Graphs

- An **OR Node** is the choice of an **action** at the state;
- An **AND Node** is the **outcomes** of an action;



- AO* is a **best-first search** algorithm similar to A*, but for (**acyclic**) AND/OR Graphs;
- AO* solves a state-space search problem by gradually building a solution graph G' ;
- **Input:** A non-deterministic state-space search problem;
- **Output:** A solution that has a conditional structure (a **policy**), or more generally, an **acyclic graph**.

- **Expanding Stage:** Expand one or more nodes on the fringe of the best solution graph;
- **Cost Revision Stage:** Revise and propagate the new values on the fringe using a *backward induction*;

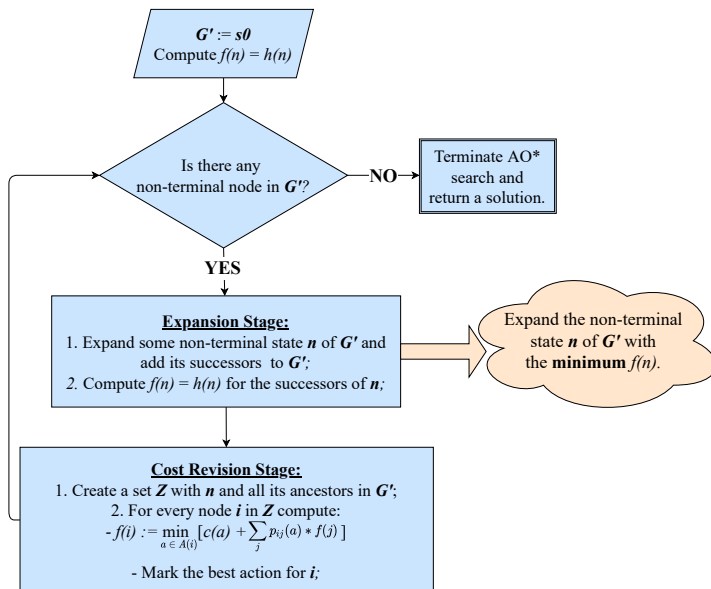
AO* Search: Expanding Stage

- Expand one or more non-terminal nodes of the best solution graph G' , namely;
 - The non-terminal node in G' with the **minimum** $f(s) = h(s)$;

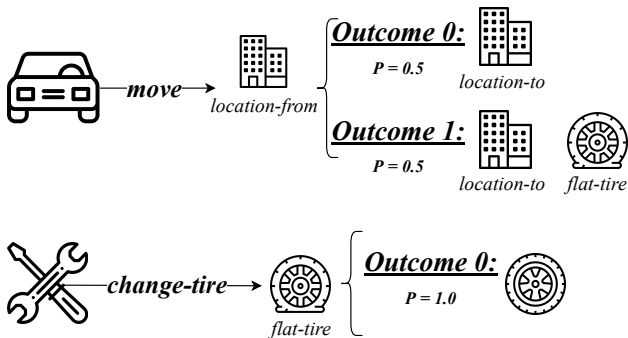
AO* Search: Cost Revision Stage

- Revise and propagate the new values on the fringe using a *backward induction*;
 - ① **Update** the *evaluation function* $f(n)$ for the expanded node n and its ancestors in G' .
 - $Z = n$ and its ancestors in G' :
$$\forall i \in Z \ f(i) := \min_{a \in A(i)} [c(a) + \sum_j p_{ij}(a) * f(j)]$$
 - ② **Mark the best action** for every OR node in G' according to $f(n)$;

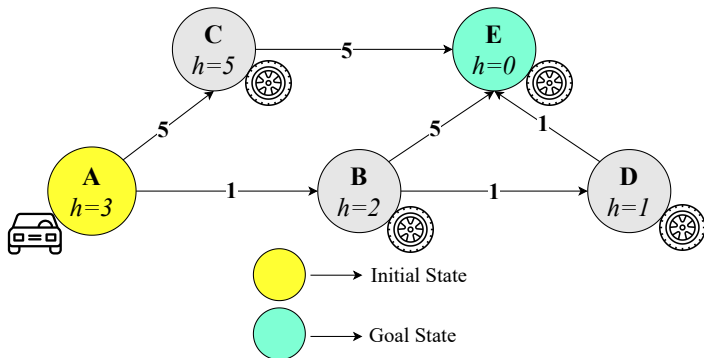
AO* Search Algorithm



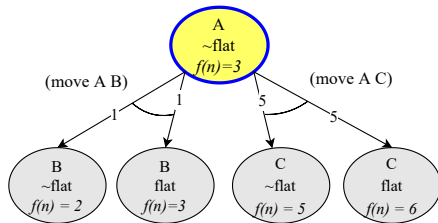
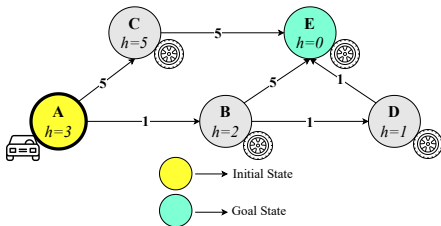
AO* Search Example (1 of 5)



AO* Search Example (2 of 5)



AO* Search Example (3 of 5)



- $G' = \langle [A, \neg f] \rangle$
- **Expansion:** $n = \langle [A, \neg f] \rangle$
- **Cost Revision:** $Z = n$ and its ancestors in G'

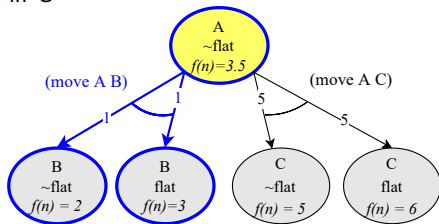
$$\min_{a \in A(i)} [c(a) + \sum_j p_{ij}(a) * f(j)]$$

- (move A B):

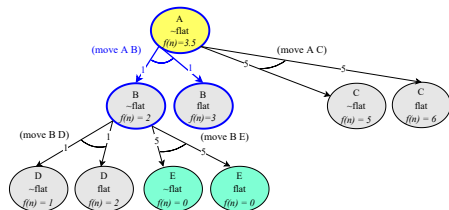
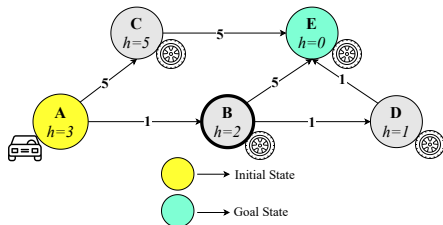
$$1 + [(0.5*3) + (0.5*2)] = 3.5$$

- (move A C):

$$5 + [(0.5*5) + (0.5*6)] = 11.5$$



AO* Search Example (4 of 5)



- $G' = \langle [A, \neg f], [B, \neg f], [B, f] \rangle$

- Expansion:** $n = \langle [B, \neg f] \rangle$

- Cost Revision:** $Z = n$ and its ancestors in G'

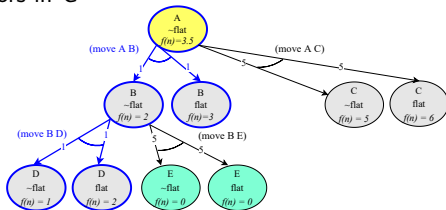
$$\min_{a \in A(i)} [c(a) + \sum_j p_{ij}(a) * f(j)]$$

- (move B D):

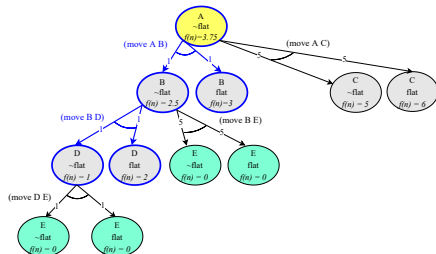
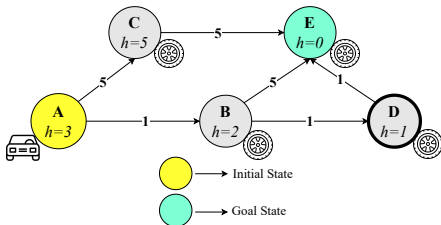
$$1 + [(0.5 * 1) + (0.5 * 2)] = 2.5$$

- (move B E):

$$5 + [(0.5 * 0) + (0.5 * 0)] = 5$$



AO* Search Example (5 of 5)



- $G' = \langle [A, \neg f], [B, \neg f], [B, f], [D, \neg f], [D, f] \rangle$

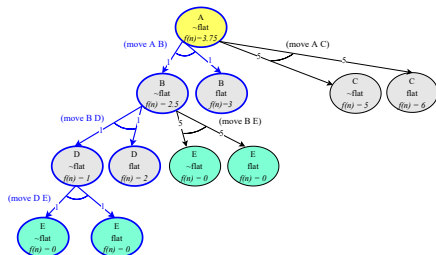
- Expansion:** $n = \langle [D, \neg f] \rangle$

- Cost Revision:** Z

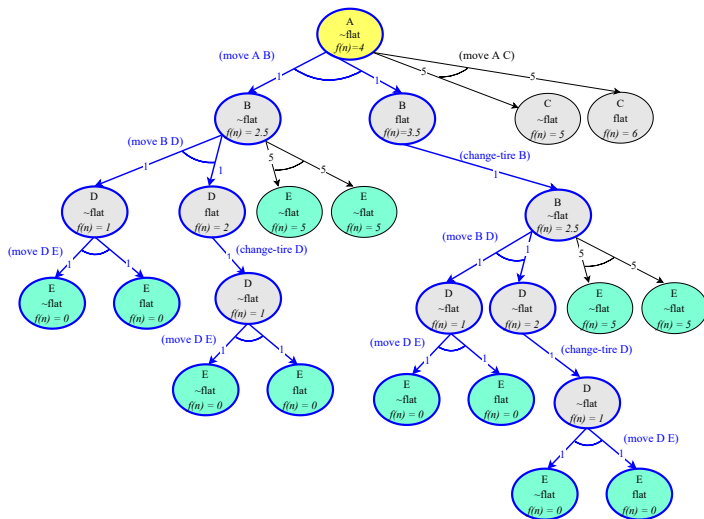
$$\min_{a \in A(i)} [c(a) + \sum_j p_{ij}(a) * f(j)]$$

- (move D E):**

$$1 + [(0.5 * 0) + (0.5 * 0)] = 1$$



AO* Search Example - Solution



Theoretical Properties of AO* Search

Theorem (Termination and Completeness)

AO search algorithm is complete and terminate if:*

- 1 *The branching factor is finite; and*
- 2 *The action costs are non-negative.*

Theorem (Optimal Efficiency)

AO search algorithm is optimal if:*

- 1 **Admissibility:** *$h(s)$ must be an admissible heuristic; and*
- 2 **Consistency:** *h must be consistent (monotone).*

- The complexity of the **Cost Revision Stage** is linear in the number of states.

AND/OR Search vs. Policy Iteration and Value Iteration

- AND/OR Search algorithms **do not explore the entire state-space**;
 - However, we need a **“good”** heuristic function.
- Policy Iteration and Value Iteration⁷ **explore the entire state-space**;

⁷ Algorithms use for solving MDPs.

What if we have cycles?! LAO* Search

- LAO* generalizes AO* for AND/OR graphs with cycles⁸:
 - It uses the same **Expansion Stage** of AO*;
 - But it uses a **Policy Evaluation** (i.e., Policy Iteration or Value Iteration) for the **Cost Revision Stage**;

⁸ Hansen and Zilberstein, *Heuristic Search in Cyclic AND/OR Graphs*. AAAI, 1998.

Take-Away Message

- *Heuristic Search* **tends to not explore** the entire state-space to find a solution (plan);
- We need **admissible heuristics** for **optimal** planning;
- The use of **non-admissible heuristics** may be **faster** to find a solution, but such a solution may **not be optimal**;

Why AI Planning is so interesting?!

- The most effective *Planning* algorithms **DO NOT** rely on data or use *Learning*;
- *Symbolic Planning* is **MUCH MORE** effective than *Learning* techniques for several problems:
 - Atari Games⁹;

⁹ Lipovetzky, Ramírez, and Geffner, *Classical Planning with Simulators: Results on the Atari Video Games*, IJCAI, 2015.

Thank you!
`pereira@diag.uniroma1.it`